COMPREHENSIVE THESIS DEFENSE Q&A GUIDE

Master of Science in Information Technology (M.Sc. IT / MIT)

Institute of Science and Technology (IoST), Tribhuvan University, Nepal

Thesis Title: *Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition*

Candidate: Erize Rai | **Roll No:** 8025006 | **Reg. No:** 5-3-28-193-2023
Supervisor: Asst. Prof. Dr. Dhiraj Kedar Pandey

Contents

1 Category 1: Computer Vision & Mathematical Foundations 2

2 Category 2: Theoretical Framework & Paradigm Comparison (SSL vs. VLM) 5

3 Category 3: Your Novel Contribution — HAF-STD Net 7

4 Category 4: Methodology, Datasets & Implementation Details 9

5 Category 5: Experimental Results, Metrics & Statistical Analysis 11

6 Category 6: Examiner Traps, Counter-Questions & Defense Strategy 13

1 Category 1: Computer Vision & Mathematical Foundations

Q1. How many bits are in a color image? What does "bit depth" mean?

Answer:

- **Bit Depth (Color Depth):** The number of binary bits utilized to specify the color or intensity of a single individual pixel.
- **Binary Image (1-bit):** $2^1 = 2$ states (0 = black, 1 = white).
- **Grayscale Image (8-bit):** Single channel with 8 bits per pixel, yielding $2^8 = 256$ intensity levels (0 to 255).
- **Standard True Color Image (24-bit RGB):** Composed of three independent channels: Red, Green, and Blue. Each channel has an 8-bit depth ($3 \times 8 = \mathbf{24}$ bits per pixel).
- **Total Possible Colors:**

$$2^{24} = 16,777,216 \text{ distinct colors } (\approx 16.78 \text{ million colors})$$

- **RGBA (32-bit):** 24-bit true color + 8-bit Alpha channel (transparency).
- **High Dynamic Range (HDR):** 10-bit ($2^{30} \approx 1.07$ billion), 12-bit, or 16-bit float per channel.

Q2. How is a video represented mathematically as a digital tensor?

Answer:

- **Static Grayscale Image:** 2D matrix $\mathbf{I} \in \mathbb{R}^{H \times W}$, where H is height and W is width.
- **Static Color (RGB) Image:** 3D tensor $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ (or in PyTorch channel-first format: $\mathbb{R}^{C \times H \times W}$), where $C = 3$.
- **Video Clip (Spatiotemporal Volume):** 4D tensor adding the temporal frame index T:

$$\mathbf{V} \in \mathbb{R}^{T \times H \times W \times C} \quad \text{or} \quad \mathbf{V} \in \mathbb{R}^{C \times T \times H \times W}$$

- **Batch of Videos (Neural Network Input):** 5D tensor:

$$\mathbf{X} \in \mathbb{R}^{B \times C \times T \times H \times W}$$

where B is batch size. In your thesis, V-JEPA 2 takes $T = 8$ uniform frames resized to 224×224 with $C = 3$.

Q3. What is the exact mathematical formula and step-by-step derivation of PCA?

Answer: Given a dataset matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ (N samples, D original feature dimensions):

1. **Step 1: Mean Centering (Standardization):** Compute the empirical feature-wise mean vector:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_i \in \mathbb{R}^{1 \times D}, \quad \tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \mu$$

2. **Step 2: Sample Covariance Matrix:** Compute the $D \times D$ symmetric covariance matrix

$\boldsymbol{\Sigma}$:

$$\boldsymbol{\Sigma} = \frac{1}{N-1}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} \in \mathbb{R}^{D\times D}$$

where element $\boldsymbol{\Sigma}_{jk} = \mathrm{Cov}(X_j, X_k)$ quantifies pairwise variance and co-dependence.

3. **Step 3: Eigen-Decomposition:** Solve the eigenvalue problem for $\boldsymbol{\Sigma}$:

$$\boldsymbol{\Sigma}\mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad \text{subject to } \|\mathbf{v}_j\|_2 = 1, \quad \mathbf{v}_j^T\mathbf{v}_k = 0 \ (\forall j \neq k)$$

where λ_j is the variance captured along the principal axis eigenvector $\mathbf{v}_j$.

4. **Step 4: Sorting and 95% Variance Retention Criterion:** Order eigenvalues in descending order: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D$. Find minimal cutoff k^*:

$$k^* = \arg\min_k \left(\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{D} \lambda_j} \geq 0.95 \right)$$

5. **Step 5: Projection to Uncorrelated Subspace:** Form the projection matrix $\mathbf{W}_{k^*} = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_{k^*}] \in \mathbb{R}^{D\times k^*}$. The dimensionally reduced representation is:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_{k^*} \in \mathbb{R}^{N\times k^*} \quad (k^* \ll D)$$

Q4. Why is StandardScaler strictly required prior to fitting PCA?

Answer: PCA is an unsupervised variance-maximization technique. If features exist on different numerical scales, features with arbitrarily larger numerical variance will artificially dominate the first few eigenvectors $\mathbf{v}_j$, irrespective of their true informational content. Applying standard normalization:

$$\tilde{z}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j}$$

ensures every feature dimension has zero mean ($\mu = 0$) and unit variance ($\sigma^2 = 1$), preventing scale-induced distortion of principal components.

Q5. What is the mathematical formula for the 1D Haar Discrete Wavelet Transform (DWT)?

Answer: Given a 1D discrete temporal sequence $\mathbf{h} = [h_0, h_1, \ldots, h_{T-1}]$ along the video frame dimension:

- **Low-Frequency Approximation (Appearance/Trend stream):**

$$a_t = \frac{h_{2t} + h_{2t+1}}{2}$$

Captures static, slowly-varying background and visual context.

- **High-Frequency Detail (Motion Dynamics stream):**

$$d_t = \frac{h_{2t} - h_{2t+1}}{2}$$

Captures temporal differences, sudden velocity shifts, and transient motion between con-

secutive frames.

- **Computational Advantage:** Haar DWT provides time-frequency localization in $\mathcal{O}(T)$ linear time with **zero learnable parameters**, ideal for short temporal tokens ($T = 8$).

Q6. What is Cosine Similarity vs. Euclidean Distance, and why is a Cosine Head used in HAF-STD Net?

Answer:

- **Euclidean Distance:** $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum (u_i - v_i)^2}$. Sensitive to the magnitude/norm of embeddings.
- **Cosine Similarity:** $\text{CosSim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^T \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \cos(\theta)$. Measures directional alignment on the unit hypersphere, strictly invariant to vector norm.
- **Why Cosine Classification Head:**

$$p(y = c \mid \mathbf{f}) = \frac{\exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_c\right)}{\sum_{j=1}^{C} \exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_j\right)}, \quad \text{where } \hat{\mathbf{f}} = \frac{\mathbf{f}}{\|\mathbf{f}\|_2}, \ \hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$$

In deep self-supervised embedding spaces, semantic class identity is encoded in the **angular orientation** of feature vectors, while norm variations can occur from LayerNorm and pooling. Normalizing features and class prototypes with a learnable temperature scale τ stabilizes gradient propagation and prevents saturation.

2 Category 2: Theoretical Framework & Paradigm Comparison (SSL vs. VLM)

Q7. What is the fundamental difference between SSL (V-JEPA 2) and VLM (Qwen2.5-VL)?

Answer:

- **Self-Supervised Learning (V-JEPA 2):** Trained purely on unlabelled video data using predictive visual pretext tasks (masked latent feature prediction). It learns intrinsic visual-geometric, motion, and temporal dynamics without linguistic supervision.
- **Vision-Language Modeling (Qwen2.5-VL):** Trained multimodally on paired image/video-text descriptions via autoregressive next-token prediction. Features are aligned with language semantic tokens, prioritizing nameable objects and static scene context over low-level motion nuances.

Q8. What is JEPA, and how does it differ from generative models and contrastive models?

Answer:

- **Generative Models (e.g., VideoMAE):** Reconstruct masked pixels in image space ($\min \|\hat{I} - I\|$). Pixel reconstruction wastes significant network capacity predicting high-frequency visual noise (e.g., grass blades, arena lighting variations) that has no relevance to semantic action recognition.
- **Contrastive Models (e.g., SimCLR, CLIP):** Enforce invariance by pulling positive views together and pushing negative views apart using InfoNCE loss. They require extensive data augmentations and large negative sample batches, which can be computationally prohibitive.
- **JEPA (Joint-Embedding Predictive Architecture):** Predicts missing target representations **directly in latent feature space**:

$$\min_{\theta,\phi} \|P_\phi(\Delta_y, E_\theta(x)) - \mathrm{sg}(E_{\bar{\theta}}(y))\|_1$$

It discards unpredictable pixel-level noise while preserving high-level semantic dynamics.

Q9. How does JEPA prevent representation collapse without negative pairs?

Answer: Contrastive learning relies on negative pairs to prevent all outputs collapsing to a constant vector. JEPA prevents collapse through three architectural mechanisms:

1. **Stop-Gradient Operator $\mathrm{sg}(\cdot)$:** Applied to the target encoder $E_{\bar{\theta}}(y)$, preventing gradients from pulling targets toward zero.
2. **Exponential Moving Average (EMA):** The target encoder weights $\bar{\theta}$ are updated slowly via EMA:
$$\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta$$
3. **Predictor Information Bottleneck:** The predictor network P_ϕ operates only on the context representation $E_\theta(x)$ with mask tokens Δ_y, making trivial constant collapse non-viable.

Q10. What is the mathematical objective function of V-JEPA 2 and V-JEPA 2.1?

Answer:

- **V-JEPA 2 (Global Mask-Denoising Latent Loss):**

$$\mathcal{L}_{\text{V-JEPA 2}} = \|P_\phi(\Delta_y, E_\theta(x)) - \text{sg}(E_{\bar{\theta}}(y))\|_1$$

  where x is context tokens, y is the target video, Δ_y are mask tokens, and sg is stop-gradient.

- **V-JEPA 2.1 (Dense Per-Token Prediction Loss):**

$$\mathcal{L}_{\text{predict}} = \frac{1}{|M|} \sum_{i \in M} \|P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i)\|_1$$

  where M denotes the set of masked spatiotemporal token indices and i indexes individual tokens. Multi-layer intermediate supervision ensures that intermediate encoder layers also develop rich semantic features.

Q11. Explain the architecture of Qwen2.5-VL and the role of MRoPE.

Answer:

- **Vision Backbone:** ViT with 3D Convolution patch embedding layer (Conv3D) and window-partitioned attention, allowing native variable-resolution and dynamic FPS frame sampling.
- **Decoder:** 7B parameter autoregressive Transformer LM decoder.
- **MRoPE (Multimodal Rotary Position Embedding):** Decomposes rotational position encoding into three independent geometric dimensions:

$$\mathbf{R}_{\Theta,m} = \mathbf{R}_{\Theta_{\text{temporal}},t} \oplus \mathbf{R}_{\Theta_{\text{vertical}},h} \oplus \mathbf{R}_{\Theta_{\text{horizontal}},w}$$

  This provides spatial coordinate grounding and temporal sequence synchronization for interleaved video and text tokens.

3 Category 3: Your Novel Contribution — HAF-STD Net

Q12. What is HAF-STD Net, and what was the core motivation behind it?

Answer:

- **Full Name:** **H**ierarchical **A**daptive **F**usion with **S**pectral-**T**emporal **D**ecomposition **Net**work.
- **Core Motivation:** Standard probing methods discard critical signals by applying global temporal pooling on the final encoder layer:
  1. *Temporal Dynamic Loss:* Global pooling flattens motion transients into static averages.
  2. *Depth Hierarchy Loss:* Intermediate layers encode low-level kinematic motion, whereas the final layer concentrates on abstract visual semantics.
- **Proposed Solution:** HAF-STD Net extracts 4 intermediate layers (at 25%, 50%, 75%, and 100% depth) and decomposes them into 12 distinct feature streams using Haar wavelets and inter-layer velocity.

Q13. Walk through the complete mathematical workflow of HAF-STD Net.

Answer:

1. **Input:** A 4D spatiotemporal tensor $\mathbf{X} \in \mathbb{R}^{B\times 4\times T\times 1280}$ extracted from 4 encoder layers across T frames.
2. **Component 1: Spectral-Temporal Decomposition via 1D Haar DWT:** For each layer $k \in \{0, 1, 2, 3\}$, decompose temporally:

$$\mathbf{a}_k = \frac{1}{2}(\mathbf{h}_{2t} + \mathbf{h}_{2t+1}) \quad \text{(Appearance / Low-Frequency Trend)}$$

$$\mathbf{d}_k = \frac{1}{2}(\mathbf{h}_{2t} - \mathbf{h}_{2t+1}) \quad \text{(Motion Dynamics / High-Frequency Detail)}$$

3. **Component 2: Depth Representation Velocity:** Compute inter-layer delta between consecutive layers:

$$\Delta\mathbf{H}_k = \bar{\mathbf{H}}_{k+1} - \bar{\mathbf{H}}_k \quad \text{for } k \in \{0, 1, 2\}, \quad \Delta\mathbf{H}_3 = \mathbf{0}$$

   where $\bar{\mathbf{H}}_k$ is the temporally mean-pooled representation at layer k.
4. **Component 3: Dedicated Projection Heads ($4 \times 3 = 12$ Streams):** Each stream $\mathbf{s}_i \in \mathbb{R}^{1280}$ is projected into $\mathbb{R}^{256}$:

$$\mathbf{s}'_i = \text{Dropout}(\text{GELU}(\text{LayerNorm}(\mathbf{W}_i\mathbf{s}_i + \mathbf{b}_i)))$$

5. **Component 4: Adaptive Routing Network:** Concatenate all 12 projected streams into a

vector of length $12 \times 256 = 3072$, passed through a router MLP with softmax:

$$\mathbf{w} = \text{Softmax}\left(\text{MLP}([\mathbf{s}'_1; \mathbf{s}'_2; \ldots; \mathbf{s}'_{12}])\right) \in \mathbb{R}^{12}, \quad \mathbf{f} = \sum_{i=1}^{12} w_i \cdot \mathbf{s}'_i \in \mathbb{R}^{256}$$

6. **Component 5: Cosine Similarity Classification Head:**

$$\text{Logits} = \tau \cdot \left(\frac{\mathbf{f}}{\|\mathbf{f}\|_2}\right) \left(\frac{\mathbf{W}_{\text{class}}}{\|\mathbf{W}_{\text{class}}\|_2}\right)^T$$

where τ is a learnable scaling factor initialized to 16.0.

Q14. Why did HAF-STD Net bypass PCA while other baseline classifiers used PCA?

Answer: PCA requires flattening multi-dimensional features into a 2D matrix $(N \times D_{\text{total}})$, which destroys the ordered temporal dimension (T) and discrete layer-depth indexing $(K = 4)$.

- HAF-STD Net strictly depends on the structured 4D tensor $(B, K = 4, T, D = 1280)$ to compute temporal differences along time (T) and velocity deltas across layer depth (K).
- In contrast, baseline classifiers (MLP, SVM, Random Forest) operate on flat vectors and require PCA to reduce dimensionality, eliminate multicollinearity, and prevent overfitting.

Q15. What is the parameter footprint and training efficiency of HAF-STD Net?

Answer:

- HAF-STD Net introduces only **4.8 million trainable parameters**.
- Relative to V-JEPA 2's backbone (654M parameters), HAF-STD Net adds **less than 0.74% overhead**.
- It trains in under 2 minutes with early stopping on a single T4 GPU, providing an efficient, lightweight downstream classification head.

4 Category 4: Methodology, Datasets & Implementation Details

Q16. What is the Frozen-Feature Protocol, and why was it chosen over full fine-tuning?

Answer:

- **Protocol Definition:** Backbone weights Θ_{frozen} remain locked throughout downstream training ($\nabla_{\Theta}\mathcal{L} = \mathbf{0}$). Only the downstream probe weights $\mathbf{W}_{\text{head}}$ are optimized.
- **Two Primary Reasons:**
  1. *Isolates Pretrained Representation Quality:* End-to-end fine-tuning modifies backbone weights, making it unclear whether performance gains come from the pretraining paradigm or downstream tuning hyperparameters. Frozen evaluation directly measures intrinsic feature quality.
  2. *Hardware Feasibility:* Full fine-tuning of a 7B VLM and 654M ViT-Huge requires multi-node A100 clusters, exceeding the single T4 GPU resource budget.

Q17. Detail the two datasets used. Why choose both FineGym and HMDB51?

Answer:

Dataset	Clips	Classes	Granularity	Key Challenge
FineGym	1,200	4	Fine-Grained	Identical backgrounds, athlete attire; subtle kinematics
HMDB51	6,754	51	Coarse-Grained	Diverse scenes, lighting, camera angles, distinct actions

Rationale: Evaluating both datasets allows testing whether model advantages persist across both **fine-grained kinematic actions** (where scene context is uninformative) and **coarse-grained actions** (where static scene and object cues are prominent).

Q18. What was your hardware infrastructure and software stack?

Answer:

- **Hardware:** Kaggle Cloud environment, 1× NVIDIA Tesla T4 GPU (15.6 GB VRAM), Intel Xeon @ 2.20 GHz (2 vCPUs), 13 GB RAM.
- **Software:** Python 3.10, PyTorch 2.1.2 with CUDA 12.1, Hugging Face Transformers 4.38.0, BitsAndBytes 0.42.0 (NF4 quantization), scikit-learn 1.4.1, OpenCV 4.9.0.

Q19. What is 4-bit NF4 Quantization, and why was it necessary for Qwen2.5-VL?

Answer:

- **Why Necessary:** Qwen2.5-VL has 7 billion parameters. In standard 16-bit float ('fp16'), weights alone require:

$$7 \times 10^9 \times 2 \text{ bytes} \approx 14 \text{ GB VRAM}$$

Adding video input tokens and activation memory immediately causes Out-Of-Memory (OOM) on a 15.6 GB T4 GPU.

- **What is NF4 (NormalFloat4):** An information-theoretically optimal quantile quantization data type designed for normally distributed neural network weights (from QLoRA). It compresses weights to 4 bits per parameter, reducing the model footprint to ~ 4 GB while retaining high fidelity.

Q20. What prompt and chat template were used for Qwen2.5-VL feature extraction?

Answer: Frames were passed to the processor using a neutral prompt with no leading task bias:

```
messages = [{'role': 'user', 'content': [
    {'type': 'video', 'video': frames},
    {'type': 'text', 'text': 'Describe.'}
]}]
```

The model ran with `output_hidden_states=True`, extracting the final hidden state ($D = 3584$) averaged across visual-text tokens.

Q21. Explain the training regularization techniques used: Label Smoothing and Mixup.

Answer:

- **Label Smoothing ($\epsilon = 0.1$):** Softens one-hot target vectors to prevent overconfidence:

$$y_c^{\text{smooth}} = (1 - \epsilon)y_c + \frac{\epsilon}{C}$$

- **Feature-Space Mixup ($\alpha = 0.2$):** Synthesizes virtual training samples by taking convex combinations of feature vectors and targets:

$$\tilde{\mathbf{z}} = \lambda \mathbf{z}_i + (1 - \lambda)\mathbf{z}_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha)$$

This regularizes decision boundaries and mitigates overfitting on frozen feature spaces.

Q22. How did you ensure zero data leakage during feature preprocessing?

Answer: Data was first partitioned into **80% training** and **20% evaluation** splits. The `StandardScaler` and `PCA` transformations were **fitted strictly on the 80% training split**. The resulting scaling statistics and projection matrices were then applied to transform the 20% evaluation split. The evaluation split was never seen during preprocessing fitting.

5 Category 5: Experimental Results, Metrics & Statistical Analysis

Q23. What are the headline quantitative results of your thesis?

Answer:

Comprehensive Comparison of Best Performing Models across Benchmarks

Dataset	Metric	V-JEPA 2 (HAF-STD Net)	Qwen2.5-VL (DeepMLP)	Margin
FineGym (4-class)	**Accuracy**	**98.1%**	89.1%	**+9.0%**
	Macro F1	**97.8%**	88.4%	**+9.4%**
	MCC	**0.974**	0.855	**+0.119**
	Kappa	**0.974**	0.854	**+0.120**
HMDB51 (51-class)	**Top-1 Acc**	**84.7%**	77.1%	**+7.6%**
	Top-5 Acc	**95.8%**	93.0%	**+2.8%**
	Macro F1	**85.3%**	79.1%	**+6.2%**
	MCC	**0.843**	0.765	**+0.078**
	Kappa	**0.842**	0.765	**+0.077**

Across all 12 classifiers and both benchmarks, **V-JEPA 2 outperforms Qwen2.5-VL on every single evaluation metric**.

Q24. How did HAF-STD Net perform compared to other baseline classifiers?

Answer:

- **FineGym:** HAF-STD Net achieved **98.1% accuracy** and **97.8% F1**, outperforming ResMLP (97.9% / 97.5%), DeepMLP (97.1% / 96.8%), and Logistic Regression (94.2% / 93.9%).
- **HMDB51:** HAF-STD Net achieved **84.7% Top-1 accuracy** and **85.3% Macro F1**, outperforming the best neural baselines (DeepMLP and ResMLP at 82.9% / 83.6%) by **+1.7% F1**.
- This demonstrates that multi-layer decomposition into appearance/motion wavelets and depth velocity provides meaningful discriminative gains over standard single-layer pooling.

Q25. Why did V-JEPA 2 outperform Qwen2.5-VL despite having 10x fewer parameters?

Answer:

1. **Task Specialization:** V-JEPA 2 (654M) was pretrained specifically for video understanding via masked spatiotemporal prediction in latent space, forcing the model to learn physics, motion dynamics, and temporal continuity.
2. **Linguistic Bias in VLMs:** Qwen2.5-VL (7B) aligns visual features with natural language. Text descriptions naturally emphasize static objects and scenes ("athlete on beam"), but rarely encode millisecond kinematic rotations or joint velocities.
3. **Temporal Sampling:** V-JEPA 2 natively processed 8 uniform frames per clip, whereas Qwen2.5-VL processed 4 frames due to context window and memory constraints.

Q26. Why was V-JEPA 2's margin larger on FineGym (+9.4% F1) than on HMDB51 (+6.2% F1)?

Answer:

- **FineGym is Fine-Grained:** All 4 classes occur in visually homogeneous gymnastics halls with identical apparel. Static scene cues provide zero discriminative signal; classification depends entirely on fine kinematic limb movement and trajectory over time, strongly favoring V-JEPA 2.
- **HMDB51 is Coarse-Grained:** Classes occur in distinct visual settings (e.g., swimming pool vs. horse field vs. city street). Static background and object semantics—which Qwen2.5-VL captures well—partially compensate for weaker temporal modeling, narrowing the margin.

Q27. Why did Conv1DNet perform poorly on Qwen2.5-VL features on HMDB51 (11.5% Top-1)?

Answer: `Conv1DNet` applies 1D convolutions with local receptive fields (kernel size 3 or 5) across adjacent feature indices. In convolutional feature maps, adjacent pixels have spatial correlation. In a 3,584-dimensional latent embedding from an LLM, adjacent feature dimensions **have no spatial or physical neighborhood locality**. Convolving across arbitrary feature dimensions imposes an invalid inductive bias, hindering effective optimization.

Q28. What are MCC and Cohen's Kappa, and why are they included alongside Accuracy?

Answer:

- **Matthews Correlation Coefficient (MCC):** Evaluates all four quadrants of the multiclass confusion matrix (TP, TN, FP, FN) in balanced proportion. Values range from -1 (complete disagreement) to $+1$ (perfect prediction):

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- **Cohen's Kappa (κ):** Measures inter-rater agreement adjusted for agreement expected purely by random chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is observed accuracy and p_e is chance-expected agreement. Both metrics show near-perfect agreement ($\kappa = 0.974$ on FineGym, $\kappa = 0.842$ on HMDB51), confirming classification robustness.

6 Category 6: Examiner Traps, Counter-Questions & Defense Strategy

Q29. "Is it fair to compare V-JEPA 2 at 16-bit precision against Qwen2.5-VL at 4-bit precision?"

Answer:

- **Acknowledge the practical constraint:** 4-bit NF4 quantization was necessary to fit the 7B parameter model onto the 15.6 GB T4 GPU, as documented in Section 1.6 (Limitations).
- **Cite empirical literature:** Extensive research on NF4 quantization (Dettmers et al., QLoRA) demonstrates that NF4 retains over 98–99% of full 16-bit float representation quality.
- **Highlight the margin size:** The **9.4% F1 gap** on FineGym and **6.2% F1 gap** on HMDB51 far exceed any typical 1–2% quantization drop, demonstrating that the performance divergence is primarily driven by architectural and pretraining differences rather than quantization.

Q30. "Why didn't you apply HAF-STD Net to Qwen2.5-VL features as well?"

Answer:

- HAF-STD Net relies on two key architectural properties:
  1. Extracting per-frame spatial representations across multiple intermediate encoder layers to compute depth velocity $\Delta \mathbf{H}_k$.
  2. Operating on an uncompressed temporal sequence of frames.
- In Qwen2.5-VL, extracting multi-layer intermediate 4D tensors for 6,754 videos exceeded the 15.6 GB VRAM budget due to the 7B decoder and dynamic token compression.
- Furthermore, V-JEPA 2.1 was pretrained with explicit intermediate-layer supervision, making multi-layer representations well-suited for HAF-STD Net.

Q31. "Why did you evaluate only 4 classes in FineGym instead of 288 or 530 classes?"

Answer:

- **Hierarchy Level:** FineGym has three annotation levels: Event (4 classes), Set (288 classes), and Element (530 classes). Event-level routines span several seconds, providing sufficient temporal duration to evaluate temporal dynamics.
- **Dataset Balance:** It allowed downloading a strictly balanced benchmark of 300 verified clips per apparatus category (1,200 total).
- **Granularity Contrast:** Evaluating 4 fine-grained classes on FineGym versus 51 coarse-grained classes on HMDB51 provided a clear experimental contrast to study task granularity.

Q32. "If you had 6 more months and an A100 GPU cluster, what would you do next?"

Answer:

1. **Full Precision & Fine-Tuning Study:** Evaluate Qwen2.5-VL in `bfloat16` without quantization, and perform Parameter-Efficient Fine-Tuning (LoRA) to examine if VLMs

close the performance gap under task-specific adaptation.

2. **Cross-Paradigm Fusion Architecture:** Implement a multimodal cross-attention head fusing V-JEPA 2's temporal representations with Qwen2.5-VL's language-grounded semantic tokens.
3. **FineGym Sub-Action Granularity:** Extend HAF-STD Net to FineGym's Element-level (530 classes) to evaluate split-second acrobatics and gymnastic maneuvers.

Oral Defense Presentation Checklist

- **Whiteboard Readiness:** Be prepared to sketch the **HAF-STD Net pipeline** (4 layers $\rightarrow$ Haar DWT low/high + velocity $\rightarrow$ 12 heads $\rightarrow$ router MLP $\rightarrow$ cosine head) and write the **PCA covariance and eigenvalue equations**.
- **Key Numbers to Memorize:**
  - FineGym: V-JEPA 2 + HAF-STD Net = **98.1% Acc**, **97.8% F1**, **0.974 MCC** vs. Qwen2.5-VL = **89.1% Acc**, **88.4% F1**, **0.855 MCC** (**+9.4% F1 gap**).
  - HMDB51: V-JEPA 2 + HAF-STD Net = **84.7% Top-1**, **95.8% Top-5**, **85.3% F1**, **0.843 MCC** vs. Qwen2.5-VL = **77.1% Top-1**, **93.0% Top-5**, **79.1% F1**, **0.765 MCC** (**+6.2% F1 gap**).
  - Model Scale: V-JEPA 2 (654M params) vs. Qwen2.5-VL (7B params, $\approx 10\times$ larger).
  - HAF-STD Net Scale: **4.8M parameters** ($< 0.74\%$ of backbone size).
- **Tone:** Confident, analytical, and scientifically grounded. Frame hardware limits as pragmatic engineering design choices.